# Bài 6: RAG end-to-end với LangChain/LlamaIndex, Chunking & Đánh giá chất lượng

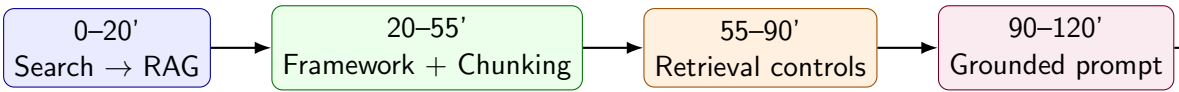
AI Engineer – Module 3: AI ứng dụng

Lecture deck – 2,5 giờ

# Mục tiêu Bài 6

- Dùng **LangChain/LlamaIndex** để ghép retrieval với LLM thành RAG end-to-end.
- Nắm các kỹ thuật **chunking**: fixed-size, recursive, semantic.
- Thiết kế prompt có grounding, citation nội bộ và quyền trả lời "không đủ thông tin".
- Phân biệt lỗi retrieval và lỗi generation.
- Đánh giá RAG bằng **faithfulness, answer relevancy, context recall** và error analysis.

# Bản đồ 2,5 giờ




## Tinh thần chính

Không đánh giá RAG bằng cảm giác “trả lời nghe hay”, mà bằng bằng chứng, bộ test và phân tích lỗi.

# Từ Bài 5 sang Bài 6

- Bài 5 đã xây nền: pipeline, chunk, embedding, vector DB và truy vấn top-k.
- Bài 6 thêm LLM để tổng hợp câu trả lời từ các chunk được retrieval, dùng framework.
- Từ "tìm đoạn liên quan" chuyển thành "trả lời có căn cứ".
- Độ khó mới: model có thể dùng sai bằng chứng hoặc bịa thêm ngoài context.

# Case mở đầu: trợ lý hỏi-đáp sổ tay vận hành

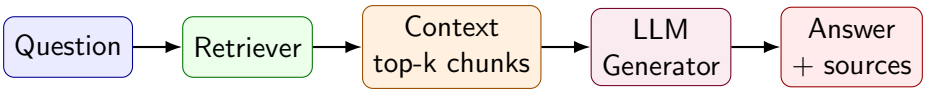
- Người dùng hỏi: "Khi site mất kết nối chập chờn, cần kiểm tra gì trước?"
- Kho tri thức: sổ tay vận hành, checklist xử lý sự cố, ticket cũ, quy trình escalation.
- Mục tiêu: trả lời ngắn, đúng nguồn, nêu khi không đủ thông tin.
- Rủi ro: trả lời tự tin nhưng sai phiên bản quy trình.

# Search thường trả về gì?




- Search giúp tìm bằng chứng, nhưng chưa tự viết câu trả lời hoàn chỉnh.
- Người dùng vẫn phải đọc, tổng hợp, loại nhiễu và kiểm tra nguồn.

# RAG thêm bước generation




## Định nghĩa thực dụng

RAG = retrieval để lấy bằng chứng + generation để diễn đạt câu trả lời dựa trên bằng chứng đó.

# RAG giải quyết vấn đề gì?

- Thêm kiến thức nội bộ mà model gốc không biết.
- Cập nhật tài liệu nhanh hơn fine-tuning.
- Cho phép truy vết nguồn khi trả lời.
- Giảm, nhưng không triệt tiêu hoàn toàn hallucination.
- Tách được lỗi tìm tài liệu và lỗi trả lời.

# RAG không phải cây đũa thần

- Nếu tài liệu sai, RAG có thể trả lời sai có nguồn.
- Nếu chunking kém, retriever không lấy đúng bằng chứng.
- Nếu prompt yếu, LLM có thể bỏ qua context hoặc suy diễn thêm.
- Nếu câu hỏi mơ hồ, top-k có thể chứa nhiều hướng diễn giải.
- Vẫn cần test set, metric và human review theo mức rủi ro.

# Ba vai trò trong một hệ RAG

- Kho tài liệu: nguồn sự thật được quản trị.
- Retriever: tìm bằng chứng phù hợp với câu hỏi.
- Generator: diễn đạt câu trả lời từ bằng chứng.
- Evaluator: kiểm tra câu trả lời có đúng, có căn cứ và có ích không.
- Guardrail: kiểm soát dữ liệu, quyền truy cập và tình huống không đủ thông tin.

# Câu hỏi nhanh

- Nếu retriever lấy sai tài liệu, generator có thể trả lời đúng không?
- Nếu retriever lấy đúng tài liệu, generator có thể trả lời sai không?
- Nếu tài liệu không có câu trả lời, hệ thống nên làm gì?
- Nếu người dùng không có quyền xem tài liệu, RAG có được dùng tài liệu đó không?

# RAG tối giản gồm những bước nào?

- Offline: thu thập tài liệu, làm sạch, chia chunk, embedding và index.
- Online: nhận câu hỏi, embedding câu hỏi, retrieval top-k, tạo prompt, gọi LLM.
- Output: câu trả lời, nguồn trích dẫn, mức tự tin hoặc cảnh báo.
- Evaluation: chạy bộ câu hỏi chuẩn và phân tích lỗi.

# Vì sao cần framework (LangChain/LlamaIndex)?

- RAG có nhiều mảnh: loader, splitter, embedding, vector store, retriever, prompt, LLM.
- LangChain và LlamaIndex cung cấp **khối dựng sẵn** và cách ghép chúng thành pipeline.
- Giúp PoC nhanh, chuẩn hóa interface, dễ đổi thành phần (đổi vector DB, đổi LLM).

**Cảnh báo**

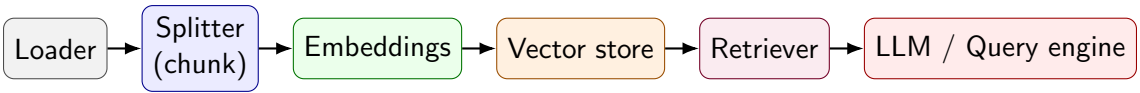
Framework không thay thế tư duy pipeline; nó che bớt chi tiết nhưng cũng dễ giấu lỗi retrieval nếu không mở ra kiểm tra.

# LangChain vs LlamaIndex

| Tiêu chí | LangChain | LlamaIndex |
|---|---|---|
| Trọng tâm | Khung ghép LLM, chains, agents, tools | Tối ưu cho index & retrieval trên dữ liệu |
| Thế mạnh | Linh hoạt orchestration, tool-use | Ingestion/index/query thuận tiện |
| RAG | Đủ khối để tự dựng pipeline | Query engine RAG sẵn, ít code |
| Khi dùng | Cần workflow/agent phức tạp | Tập trung hỏi-đáp tài liệu |

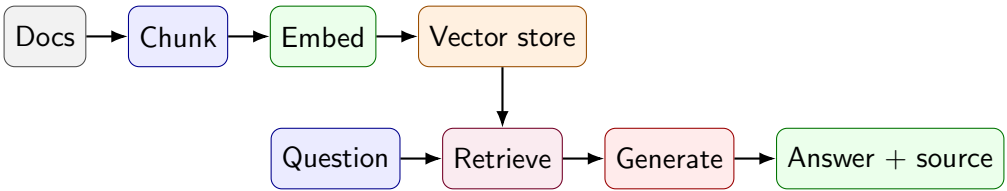
Hai framework có thể kết hợp; chọn theo nhu cầu, không chọn theo độ phổ biến. Vẫn phải tự kiểm soát chất lượng.

# Khối dựng RAG trong framework




- Mỗi khối có thể thay riêng: đổi splitter, đổi embedding model (HF Hub), đổi vector DB (FAISS/Chroma/Milvus).
- Giữ khả năng **in ra top-k chunk** để kiểm tra, đừng chỉ tin câu trả lời cuối.

# Pipeline RAG end-to-end



# Hai luồng: indexing và serving

- Indexing pipeline chạy trước hoặc chạy định kỳ khi tài liệu thay đổi.
- Serving pipeline chạy mỗi khi người dùng đặt câu hỏi.
- Lỗi ở indexing có thể ảnh hưởng lâu dài đến mọi câu trả lời.
- Lỗi ở serving thường liên quan truy vấn, top-k, prompt và generator.

# Vì sao chunking quan trọng?

- LLM không nên nhận toàn bộ kho tài liệu trong prompt.
- Retriever cần các đơn vị đủ nhỏ để tìm đúng, đủ lớn để còn ngữ cảnh.
- Chunk sai có thể cắt mất điều kiện, ngoại lệ hoặc heading quan trọng.
- Chunking quyết định trực tiếp retrieval quality và answer quality.

# Chunk quá nhỏ / quá lớn

## Chunk quá nhỏ

- Dễ mất ngữ cảnh.
- Lấy được câu riêng lẻ nhưng thiếu điều kiện áp dụng.
- Generator phải suy đoán phần còn thiếu.

## Chunk quá lớn

- Nhiễu nhiều thông tin không liên quan.
- Tốn token context.
- Retrieval có thể kém sắc nét.

# Fixed-size chunking

- Chia theo số ký tự, số token hoặc số câu gần cố định.
- Ưu điểm: dễ cài đặt, dễ kiểm soát kích thước.
- Nhược điểm: có thể cắt ngang mục, bảng, checklist hoặc đoạn điều kiện.
- Phù hợp cho PoC nhanh, cần xem lại failure case.

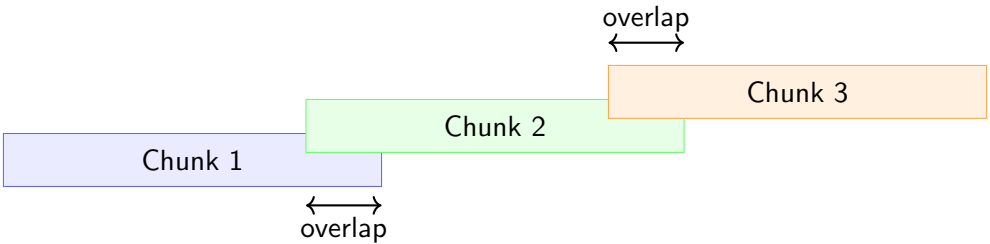
# Recursive / structure-aware chunking

- Ưu tiên chia theo heading, mục, đoạn, câu trước khi chia theo kích thước.
- Giữ cấu trúc tài liệu tốt hơn fixed-size.
- Hữu ích với sổ tay vận hành, FAQ, SOP, tài liệu có section rõ.
- Cần xử lý riêng bảng, danh sách dài và phụ lục.

# Semantic chunking – chỉ cần hiểu trực giác

- Cố gắng chia theo ranh giới ý nghĩa thay vì chỉ đếm token.
- Một chunk nên chứa một đơn vị tri thức tương đối hoàn chỉnh.
- Có thể tốt hơn với tài liệu dài, nhưng phức tạp và khó debug hơn.
- Trong lớp: hiểu ý tưởng, không cần tối ưu thuật toán.

# Chunk overlap




- Overlap giúp không mất ngữ cảnh tại ranh giới chunk.
- Overlap quá lớn làm tăng số chunk, chi phí và trùng lặp context.

# Metadata là một phần của retrieval

- Metadata ví dụ: loại tài liệu, phiên bản, ngày hiệu lực, site, thiết bị, phòng ban.
- Filter theo metadata giúp tránh lấy sai tài liệu hoặc sai phiên bản.
- Nguồn/citation nên gắn với metadata đủ để người dùng kiểm tra.
- Metadata nghèo làm RAG khó quản trị khi kho tài liệu lớn.

# Chunking checklist

- Mỗi chunk có đủ ngữ cảnh để người lạ hiểu không?
- Chunk có giữ heading/section/path của tài liệu không?
- Điều kiện, ngoại lệ và cảnh báo có bị cắt rời không?
- Bảng và checklist có bị biến dạng không?
- Có metadata để filter và cite không?

# Retrieval controls là gì?

- Các nút điều chỉnh cách hệ thống chọn bằng chứng.
- Quan trọng nhất trong PoC: top-k, similarity threshold, metadata filter.
- Nâng cao: hybrid search, reranking, query rewriting.
- Mục tiêu: đưa đúng bằng chứng vào context, không phải đưa càng nhiều càng tốt.

# top-k: lấy bao nhiêu chunk?




- top-k nhỏ: ít nhiễu, nhưng dễ bỏ sót bằng chứng.
- top-k lớn: nhiều bằng chứng, nhưng tăng token, nhiễu và nguy cơ mâu thuẫn.

# Similarity threshold

- Không phải chunk nào trong top-k cũng đủ liên quan.
- Threshold giúp loại kết quả có điểm quá thấp.
- Nếu tất cả điểm thấp, hệ thống nên nói không đủ thông tin hoặc hỏi lại.
- Threshold cần được hiệu chỉnh bằng test set, không chọn theo cảm tính.

# Metadata filter

- Chỉ tìm trong tài liệu đúng loại, đúng phiên bản hoặc đúng phạm vi quyền.
- Ví dụ: chỉ manual phiên bản 2026, chỉ tài liệu mạng truy nhập, chỉ site miền Bắc.
- Filter sai có thể làm mất câu trả lời đúng.
- Không có filter có thể làm lộ thông tin hoặc lấy tài liệu không phù hợp.

# Hybrid search

- Keyword search mạnh khi câu hỏi có mã lỗi, tên thiết bị, mã site, thuật ngữ chính xác.
- Semantic search mạnh khi người dùng diễn đạt khác tài liệu.
- Hybrid kết hợp hai tín hiệu để tăng robustness.
- Trong PoC, ít nhất nên so sánh keyword và semantic ở vài truy vấn.

# Đừng nhầm lỗi retrieval với lỗi generation

- Nếu context không chứa bằng chứng đúng: lỗi retrieval/indexing/query.
- Nếu context có bằng chứng đúng nhưng answer sai: lỗi generation/prompt.
- Nếu answer đúng nhưng không có citation: lỗi format/grounding.
- Nếu answer dùng tài liệu người dùng không có quyền xem: lỗi access control.

# Ma trận lỗi RAG

| **Triệu chứng** | **Khả năng lỗi** | **Cách kiểm tra nhanh** |
|---|---|---|
| Không lấy đúng chunk | Retrieval/chunking | Xem top-k chunk trực tiếp |
| Có chunk đúng nhưng trả lời sai | Prompt/generator | So sánh answer với context |
| Trả lời không có nguồn | Output format | Kiểm template và citation |
| Tự bịa khi thiếu dữ liệu | Grounding guardrail | Test câu hỏi không có đáp án |

# Grounded prompt là gì?

- Instruction buộc model chỉ trả lời từ context được cung cấp.
- Yêu cầu nêu nguồn cho từng ý quan trọng.
- Cho phép trả lời "không đủ thông tin" khi context không hỗ trợ.
- Cấm dùng kiến thức ngoài nếu bài toán yêu cầu câu trả lời theo tài liệu nội bộ.

# Cấu trúc grounded prompt

## Template tối giản

Bạn là trợ lý vận hành. Chỉ sử dụng CONTEXT để trả lời QUESTION.
Nếu CONTEXT không đủ, trả lời: "Không đủ thông tin trong tài liệu được cung cấp".
Trả lời ngắn gọn, liệt kê nguồn theo mã chunk/tài liệu.
CONTEXT: {retrieved_chunks}
QUESTION: {user_question}

# Prompt phải quy định hành vi khi thiếu dữ liệu

- Không đủ thông tin là một output hợp lệ, không phải thất bại.
- Cho phép hỏi lại nếu câu hỏi mơ hồ.
- Không được đoán nguyên nhân nếu context chỉ có triệu chứng.
- Không dùng cụm "có thể" để che việc thiếu bằng chứng.

# Citation nội bộ

- Citation giúp người dùng kiểm tra và giúp giảng viên debug.
- Citation nên chỉ rõ tài liệu, section, chunk hoặc đường dẫn nội bộ.
- Không cite nguồn không được dùng trong câu trả lời.
- Answer không có citation khó được tin trong nghiệp vụ rủi ro.

# Context packaging

- Không chỉ ghép raw chunk; nên thêm source, heading và thứ tự chunk.
- Giữ format dễ đọc để LLM biết đâu là bằng chứng.
- Loại chunk trùng lặp hoặc quá thấp điểm trước khi đưa vào prompt.
- Đặt câu hỏi sau context hoặc theo template nhất quán.

# Khi context có mâu thuẫn

- Có thể do tài liệu cũ và mới cùng được retrieval.
- Có thể do quy trình theo vùng/site khác nhau.
- Prompt nên yêu cầu nêu mâu thuẫn thay vì chọn bừa.
- Metadata version/date là chìa khóa để giảm mâu thuẫn.

# Guardrail tối thiểu cho RAG PoC

- Không đủ thông tin thì từ chối trả lời dứt khoát.
- Không trả lời ngoài phạm vi tài liệu.
- Không hiển thị chunk người dùng không có quyền xem.
- Log question, top-k chunks, prompt version, answer và feedback.
- Có human review với khuyến nghị hành động rủi ro.

# Evaluation: vì sao cần đo?

- Một vài demo hay không chứng minh hệ thống tốt.
- RAG có nhiều tham số; thay chunk/top-k/prompt có thể đổi chất lượng.
- Cần so sánh cấu hình bằng cùng một bộ câu hỏi.
- Không đo thì không biết cải thiện ở retrieval hay generation.

# Bộ câu hỏi kiểm thử

- Nên có 10–20 câu cho PoC nhỏ, đủ bao phủ các tình huống chính.
- Bao gồm câu hỏi có đáp án rõ, câu hỏi cần nhiều chunk, câu hỏi mơ hồ, câu hỏi không có đáp án.
- Mỗi câu cần expected evidence hoặc tài liệu nguồn mong muốn.
- Không chỉ lấy câu hỏi dễ từ chính heading.

# Metric 1: Context recall

- Trong các bằng chứng cần thiết, retriever lấy được bao nhiêu?
- Nếu context recall thấp, generator rất khó trả lời đúng.
- Có thể đánh giá thủ công ở lớp bằng cách kiểm top-k chunk.
- Câu hỏi: chunk đúng có nằm trong top-k không?

# Metric 2: Answer relevancy

- Câu trả lời có trả lời đúng câu hỏi người dùng không?
- Tránh trả lời dài nhưng lệch trọng tâm.
- Có thể chấm theo thang 0/1/2: không liên quan, một phần, đúng trọng tâm.
- Relevancy cao vẫn chưa đảm bảo có căn cứ.

# Metric 3: Faithfulness

- Các phát biểu trong answer có được context hỗ trợ không?
- Nếu answer thêm claim không có trong context: lỗi faithfulness.
- Faithfulness là trọng tâm để phát hiện hallucination trong RAG.
- Cần nhìn song song answer và retrieved context.

# Metric 4: Citation quality

- Citation có tồn tại và đúng nguồn không?
- Citation có hỗ trợ đúng claim được nêu không?
- Có cite quá rộng hoặc cite nguồn không liên quan không?
- Trong nghiệp vụ, citation sai có thể nguy hiểm như không có citation.

# Error categories

- Retrieval miss: không lấy được tài liệu đúng.
- Context noise: lấy nhiều chunk nhiễu làm answer lệch.
- Generation error: có bằng chứng đúng nhưng tổng hợp sai.
- Unsupported claim: answer bịa thêm ngoài context.
- Policy/access error: dùng dữ liệu không nên dùng.

# Bảng evaluation tối thiểu

| Question | Expected evidence | Retrieved? | Answer OK? | Failure note |
|---|---|---|---|---|
| Q1 | SOP-03/sec 2 | Yes | Yes | – |
| Q2 | Checklist alarm | No | No | chunking/retrieval miss |
| Q3 | None | No | Yes | trả lời không đủ thông tin |

# So sánh cấu hình

- Không thay nhiều biến cùng lúc nếu muốn hiểu nguyên nhân.
- Ví dụ A: chunk 300, overlap 50, top-k 3.
- Ví dụ B: chunk 600, overlap 100, top-k 5.
- So sánh trên cùng bộ 10 câu hỏi.
- Ghi lại trade-off: quality, token cost, latency, nhiễu.

# Tác động của tham số




Không có cấu hình tốt cho mọi tài liệu

# Privacy và access control trong RAG

- Retriever không được lấy tài liệu người dùng không có quyền xem.
- Embedding/index cũng cần được coi là tài sản dữ liệu.
- Log câu hỏi và context có thể chứa dữ liệu nhạy cảm.
- Không dùng tài liệu confidential trong demo công khai nếu chưa được phép.

# Khi nào RAG không phù hợp?

- Không có kho tài liệu đáng tin cậy hoặc tài liệu quá lỗi thời.
- Bài toán cần tính toán chính xác từ dữ liệu có cấu trúc hơn là hỏi tài liệu.
- Câu trả lời yêu cầu hành động thực thi, cần agent/tool và approval.
- Nhiệm vụ có thể giải quyết bằng rule/search đơn giản với chi phí thấp hơn.

# Thiết kế câu trả lời tốt

- Trả lời ngắn gọn trước, giải thích sau.
- Tách fact, giả định và bước đề xuất.
- Nêu nguồn cho từng nhóm ý chính.
- Nêu rõ điều kiện áp dụng và điều chưa đủ thông tin.
- Không dùng giọng quá tự tin khi bằng chứng yếu.

# Mini case: câu hỏi không có đáp án

- Question: "Quy trình reset thiết bị X phiên bản 2027 là gì?"
- Context chỉ có manual 2025 và 2026.
- Answer tốt: "Không đủ thông tin về phiên bản 2027 trong tài liệu được cung cấp."
- Answer xấu: suy diễn từ phiên bản 2026 mà không cảnh báo.

# Mini case: context có nhiễu

- Top-k chứa 1 chunk đúng và 4 chunk gần nghĩa nhưng sai loại thiết bị.
- Generator có thể tổng hợp thành câu trả lời pha trộn.
- Cải thiện: metadata filter theo device type, reranking, top-k thấp hơn hoặc prompt yêu cầu nêu nguồn.
- Bài học: nhiều context không đồng nghĩa nhiều chất lượng.

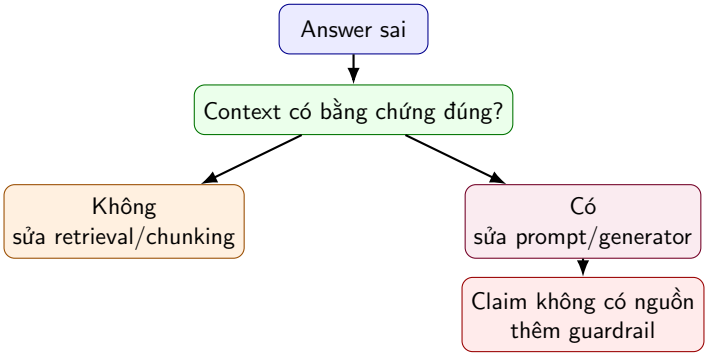
# Mini case: tài liệu đúng nhưng answer sai

- Context đã có checklist xử lý đúng.
- Answer bỏ qua bước kiểm tra an toàn hoặc đảo thứ tự thao tác.
- Cải thiện: prompt yêu cầu giữ đúng thứ tự checklist, trích nguyên bước quan trọng, human review.
- Đây là lỗi generation/prompt, không phải lỗi retrieval.

# Workflow debug RAG

- Bước 1: xem câu hỏi gốc và câu hỏi rewrite nếu có.
- Bước 2: xem top-k chunk và điểm liên quan.
- Bước 3: kiểm chunk đúng có được đưa vào context không.
- Bước 4: so answer với context, đánh dấu unsupported claims.
- Bước 5: quyết định sửa chunking, retrieval, prompt hay tài liệu.

# Debug tree


Answer sai
Context có bằng chứng đúng?
Không
sửa retrieval/chunking
Có
sửa prompt/generator
Claim không có nguồn
thêm guardrail

# Chuẩn đầu ra của một RAG PoC

- Có pipeline chạy được từ câu hỏi đến câu trả lời có nguồn (LangChain/LlamaIndex).
- Có ít nhất 10 câu hỏi test và bảng evaluation (faithfulness/relevancy/context recall).
- Có phân tích ít nhất 3 failure case.
- Có so sánh ít nhất 2 cấu hình chunk/top-k/prompt.
- Có kết luận: go, pilot, not yet hoặc cần dữ liệu tốt hơn.

# Bridge sang Lab

- Lab dùng index từ Bài 5 hoặc tạo lại nhanh.
- Ghép retrieval + grounded prompt + LLM bằng LangChain/LlamaIndex để tạo Mini RAG.
- Chạy 10 câu hỏi benchmark; thử 2 cấu hình và so sánh.
- Ghi failure case và đề xuất cải thiện.

# Knowledge check

## Nhóm 1

RAG khác semantic search ở bước nào? Khi top-k chunk không có bằng chứng đúng, lỗi nằm ở đâu? Khi answer thêm thông tin ngoài context, đó là lỗi gì?

## Nhóm 2

Khi nào nên giảm top-k? Khi nào nên tăng chunk size? Khi nào cần metadata filter? Khi nào hệ thống nên trả lời "không đủ thông tin"?

# Tổng kết Bài 6

- LangChain/LlamaIndex giúp dựng RAG end-to-end nhưng không thay thế tư duy pipeline.
- Chất lượng RAG phụ thuộc tài liệu, chunking, retrieval, prompt và evaluation.
- Luôn tách lỗi retrieval khỏi lỗi generation.
- Câu trả lời tốt cần grounded, có nguồn và biết từ chối khi thiếu bằng chứng.
- Đánh giá RAG dựa trên test set và metric faithfulness / relevancy / context recall.

# Thông điệp cuối

RAG không phải là cách làm cho LLM "biết tất cả".

RAG là cách buộc hệ thống **tìm đúng bằng chứng, trả lời có căn cứ, và bị kiểm thử bằng câu hỏi chuẩn**.